

Introduction

Every automation tool eventually runs into the same wall: the workflow finishes, and whatever it just processed is gone. n8n is no exception. A form submission comes in, triggers a workflow, maybe sends a notification — and then vanishes from memory the moment the run ends. If you've ever wondered why your "automated lead capture" workflow doesn't actually have any leads to show for it a week later, this is why.

This matters beyond n8n specifically. Persisting data — reading it, writing it, updating it safely — is one of the most fundamental skills in software engineering, and it's tested constantly in coding interviews under different names: database design questions, SQL query problems, and "what happens if two writes happen at once" concurrency questions. Learning to connect a workflow tool to a real database also happens to be one of the fastest ways to internalize core SQL concepts (`SELECT` , `INSERT` , `UPDATE`) without first having to stand up a backend from scratch.

By the end of this lesson, you'll understand why spreadsheets fail as a database, how to wire up Postgres or MySQL credentials in n8n, how to insert real records from form data, and — most importantly — the one missing clause that can quietly corrupt an entire table.

Problem Overview

Restated plainly: n8n workflows are stateless by default. A trigger fires, nodes execute in sequence, data flows through the pipeline, and when the last node finishes, none of that data is retained anywhere. If you want a record of what happened — a form submission, a lead, a support ticket — you need to explicitly write it somewhere durable.

The naive fix is a spreadsheet (Google Sheets, Airtable, etc.). That works for small volumes, but breaks down under three common conditions:

- **Concurrent writes.** Two workflow runs firing close together can race each other and corrupt rows or create duplicates, because spreadsheets have no locking or transaction guarantees.
- **Filtered reads.** Pulling "just today's leads" or "only status = new" from a spreadsheet means scanning and filtering rows manually — there's no query engine underneath.
- **Relational data.** Once you need to link leads to a separate table of follow-ups, a spreadsheet has no concept of foreign keys or joins.

The real fix is a proper database — Postgres or MySQL — accessed through n8n's dedicated nodes for each. These nodes let you query, insert, and update rows using structured operations instead of hand-rolled spreadsheet logic.

Example

Imagine a `leads` table with columns: `id`, `name`, `email`, `status`, `created_at`.

Query: "Give me only `name`, `email`, `status`, `created_at` where `status = 'new'` ." Output: a handful of fresh leads, not the entire historical table — no leads from six months ago that already closed.

Insert: A form submission triggers the workflow. The node maps `name` → the `name` column, `email` → the `email` column, and so on, pulling values straight from the trigger's incoming JSON. Output: a new row appears in `leads`, and it's still there after the workflow finishes — permanently, unlike the in-memory data that would otherwise vanish.

Update: A follow-up email goes out later, in a separate workflow run. That run needs to update the *existing* row's `status` to `contacted`, not create a new one. Output (correct): one row changes from `new` to `contacted`. Output (buggy, missing `WHERE`): every row in the table changes to `contacted`, regardless of which lead the update was meant for.

Intuition

An experienced engineer approaches this by separating three distinct operations that are often conflated by beginners: **reading**, **creating**, and **modifying** state.

- Reading (`SELECT`) should always be scoped — you almost never want an entire table, you want a specific slice of it (`WHERE status = 'new'`). Unscoped reads work, but they push filtering logic downstream where it's more error-prone and less efficient.
- Creating (`INSERT`) requires an explicit mapping between the shape of your incoming data (a webhook payload, a form submission) and the shape of your table (typed columns). This mapping is where type mismatches surface — send a number where text is expected, or omit a required field, and the database rejects it before it's ever written. That's a feature, not a bug: it stops bad data from ever reaching your table.
- Modifying (`UPDATE`) is the operation that trips people up, because it looks like `INSERT` but behaves completely differently. An `UPDATE` without a `WHERE` clause doesn't target one row — it targets *every* row that matches nothing, which in SQL means it matches everything. The mental

model that prevents this: an `UPDATE` statement is really two parts — "what to change" and "which rows qualify" — and skipping the second part doesn't mean "no rows," it means "all rows."

The other common failure mode isn't a missing clause at all — it's using the wrong node entirely. Someone needs to update a lead's status, reaches for the `Insert` node because it's the one they already know, and instead of updating the original row, n8n happily creates a second row that's supposed to represent the same lead. Now you have two records claiming to be one person, and nothing in the system tells you that happened.

Brute Force Solution

Idea: Treat every state change as a fresh insert — never update, only append new rows.

Algorithm: 1. Form submitted → insert a row. 2. Follow-up sent → insert *another* row with the same lead's info and a new status. 3. To find a lead's "current" state, query all rows for that lead and take the most recent one.

Advantages: Simple to reason about — no `WHERE` clause bugs are possible because you never modify existing rows.

Disadvantages: Table grows unbounded with duplicate data for a single logical entity. Every read now needs an extra "get latest by timestamp" step. No single source of truth for "the current state of this lead" — you've pushed the problem from write-time to read-time.

Time complexity: Insert is $O(1)$. Reading current state is $O(n)$ per lead in the worst case (or $O(\log n)$ with an index on lead ID + timestamp). **Space complexity:** $O(n \times \text{updates per lead})$ — grows with every status change, not just every unique lead.

Optimal Solution

Use `UPDATE` with a `WHERE` clause tied to a stable identifier — typically the primary key.

Step

Action

- 1 Set up Postgres/MySQL credentials once (host, database, user, password)
- 2 `SELECT` with explicit columns and a `WHERE` filter for reads
- 3 `INSERT` mapping incoming trigger fields to typed columns
- 4 `UPDATE` with `SET status = 'contacted' WHERE id = {{ $json.id }}`

The key structural idea: one row per logical entity (one row per lead), and every write to that entity after the first is an `UPDATE` scoped by the same identifier used to create it. The `id` (or another unique column) is what threads the insert and every subsequent update together as the *same* row, rather than a growing pile of near-duplicates.

Python Code

n8n's nodes generate SQL under the hood, but the same logic in raw Python (using `psycopg2` for Postgres) looks like this:

```
import psycopg2

def insert_lead(conn, name: str, email: str, status: str = "new") -> int:
    """Insert a new lead and return its generated id."""
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO leads (name, email, status)
            VALUES (%s, %s, %s)
            RETURNING id;
            """,
            (name, email, status),
        )
        lead_id = cur.fetchone()[0]
    conn.commit()
    return lead_id

def update_lead_status(conn, lead_id: int, new_status: str) -> None:
    """Update exactly one lead's status, scoped by primary key."""
    with conn.cursor() as cur:
        cur.execute(
            """
            UPDATE leads
            SET status = %s
            WHERE id = %s;
            """,
            (new_status, lead_id),
        )
    conn.commit()

def get_new_leads(conn) -> list[tuple]:
    """Fetch only leads that haven't been contacted yet."""
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT id, name, email, created_at
```

```
FROM leads
WHERE status = %s;
""",
("new",),
)
return cur.fetchall()
```

Code Walkthrough

- `insert_lead` uses parameterized placeholders (`%s`) instead of string formatting — this prevents SQL injection, the same category of bug that a missing `WHERE` clause belongs to: trusting unscoped or unsanitized input to shape your query.
- `RETURNING id` hands back the generated primary key immediately, so the caller can use it later for an update without a second lookup.
- `update_lead_status` takes `lead_id` as a required parameter, not optional — there's no code path where this function can run without a `WHERE id = %s` . That's the equivalent of n8n forcing you to map a "matching column" in the Update node's UI.
- `get_new_leads` only selects the columns actually needed, mirroring the lesson's point about not pulling `SELECT *` for a full historical table.

Dry Run

1. Form submitted with `name="Alex"` , `email="alex@example.com"` .
2. `insert_lead(conn, "Alex", "alex@example.com")` runs → row `(id=42, name="Alex", email="alex@example.com", status="new")` is written → returns `42` .
3. Days later, a follow-up email goes out. The workflow calls `update_lead_status(conn, 42, "contacted")` .
4. SQL executed: `UPDATE leads SET status = 'contacted' WHERE id = 42;` → only row `42` changes.
5. `get_new_leads(conn)` no longer returns Alex, since their status is now `contacted` , not `new` .

Complexity Analysis

- **Time:** `INSERT` is $O(1)$ amortized (assuming standard B-tree indexing on the primary key).
`UPDATE ... WHERE id = ?` is $O(\log n)$ with an index on `id` , since the database can seek

directly to the matching row instead of scanning the table. `SELECT ... WHERE status = ?` is $O(\log n + k)$ with an index on `status`, where k is the number of matching rows.

- **Space:** $O(n)$, one row per lead — no duplication, unlike the brute force append-only approach.
- These bounds hold *only* if the relevant columns are indexed and the `WHERE` clause is present. Remove the `WHERE` clause and the complexity of `UPDATE` becomes $O(n)$ — but worse than slow, it's simply wrong, since it touches every row instead of one.

Alternative Solutions

Upsert (`INSERT ... ON CONFLICT DO UPDATE` in Postgres, or `ON DUPLICATE KEY UPDATE` in MySQL) collapses insert-or-update into a single atomic statement, keyed on a unique constraint (e.g., email). This is a good fit when you don't know in advance whether a row exists yet — for example, deduplicating leads by email without a separate lookup step. Trade-off: slightly less explicit than separate insert/update logic, so it can be harder to reason about for newcomers.

Edge Cases

- **Empty result on `SELECT`:** No leads match `status = 'new'` — handle this as a valid empty state, not an error.
- **Duplicate submissions:** Same email submits the form twice — decide upfront whether that's two leads or one, and enforce it with a unique constraint if it should be one.
- **Missing required field on insert:** A required column (e.g., `email`) arrives empty — the insert should fail loudly rather than silently writing a null.
- **Update with no matching `id`:** `WHERE id = 9999` where no such row exists — the update succeeds but affects zero rows. Silent no-op, easy to miss without checking affected row count.
- **Update with no `WHERE` at all:** The catastrophic case — every row in the table is overwritten.

Common Mistakes

1. Forgetting the `WHERE` clause on an `UPDATE`, silently overwriting every row instead of one.
2. Using an `Insert` node to make what should be an `Update`, creating a duplicate row for the same entity.
3. Running `SELECT *` instead of naming needed columns and filtering with `WHERE`, pulling stale or irrelevant historical data.
4. Sending mismatched types into typed columns (a string into a numeric column), which the database will reject — but only at insert time, not earlier.

5. Not reusing saved credentials, re-entering connection details per node instead of once, increasing the chance of a typo in host/user/password.
6. Choosing a non-unique column (like `email` , without a uniqueness constraint) as the matching key for updates, risking updates hitting multiple unintended rows.

Interview Questions

- What's the difference between `UPDATE` and `UPSERT` , and when would you use each?
- How would you prevent an `UPDATE` statement from running without a `WHERE` clause in production?
- Why is a spreadsheet not a substitute for a relational database at scale?
- How do you handle two workflows trying to update the same row at the same time?
- What's the purpose of a primary key in preventing duplicate records?

Similar Problems

- **LeetCode 175: Combine Two Tables** — practices `SELECT` with joins across related tables, similar to linking leads to a follow-ups table.
- **LeetCode 176: Second Highest Salary** — reinforces writing precise, filtered `SELECT` queries instead of pulling everything.
- **LeetCode 178: Rank Scores** — builds comfort with structured queries beyond basic filtering, useful once your leads table grows more complex fields.

Key Takeaways

Data that isn't written to durable storage disappears the moment a workflow ends. A real database beats a spreadsheet once you need concurrency, filtering, or relationships. `INSERT` needs a correct field-to-column mapping; `UPDATE` needs a correct `WHERE` clause tied to a stable identifier — skip the `WHERE` , and "update one row" silently becomes "update every row."

Watch the Video

For the full walkthrough of setting up Postgres/MySQL credentials in n8n and seeing the insert/update nodes configured live, watch the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series that takes tools like n8n and actually explains how they work under the hood — not just click-through tutorials, but the reasoning behind each node, each config option, and each mistake that's easy to make once and hard to forget.

Call To Action

If this saved you from a `WHERE` -clause disaster waiting to happen, like the video on YouTube and subscribe for the rest of the n8n series. For deeper written breakdowns like this one, subscribe to the Substack. Drop a comment with what you want covered next — questions get read and answered.

Quick Review

Postgres/MySQL node: full data pull vs targeted query — when use which?

Use full pull only for small/one-off; use a `WHERE`-scoped query when data grows, avoids loading the whole table.

n8n DB nodes: what fixes stale form-submission data forever?

A database as source of truth instead of relying on execution history, which n8n purges/loses over time.

Insert node: most common failure mode?

Field names not matching real column names — mismatched mapping silently fails or errors.

Update node: what happens if you omit the match/WHERE clause?

Every row gets updated instead of the intended one — missing match condition is the classic bug.

Database node vs Spreadsheet node — when does DB win?

DB wins for concurrent writes, querying, and structured relations; spreadsheet fine only for tiny, single-user data.

Follow-up: how do credentials differ from query config in DB nodes?

Credentials (host/user/pass) are stored once and reused; query/table/column config is set per-node-operation.

n8n Lesson 27: Connecting to databases (Postgres/MySQL nodes: querying, inserting, and updating records) — free Python cheat sheet